# Programación de servicios y procesos




## Programación multiproceso

### Sincronización de procesos

# Por qué sincronizar procesos

Cuando varios procesos se ejecutan a la vez, pueden darse situaciones en que unos interfieran con los otros.

La sincronización permite:

- Compartir recursos sin conflictos: que dos procesos no escriban a la vez en el mismo archivo o base de datos.
- Mantener el orden correcto: que un proceso espere a que otro termine antes de continuar. Un proceso es dependiente de otro.
- Evitar condiciones de carrera: resultados incorrectos porque varios procesos acceden a la vez a los mismos datos. Suelen producirse cuando hay compartición incorrecta de recursos, o no se respeta el orden.

# Ejemplo de conflicto

Sean dos procesos concurrentes (A y B), que escriben letras en un mismo fichero. El primero cinco "A" y el segundo cinco veces "B". Supongamos que "A" abre el fichero antes que el proceso "B".

Podría pensarse que el fichero resultante contendría "AAAAABBBBB".

Pero, si se dan las circunstancias adecuadas, se puede producir cualquier clase de combinación. AABABBBAAB, BAABBBABAAA, BBBBBAAAAA, etc.

El SO garantiza que operaciones "write" individuales son "atómicas" (no pueden interrumpirse). Pero operaciones separadas pueden intercalarse.

Ver ejemplo en repositorio (Ejemplo1).

# Ejemplo de dependencia

Supongamos dos procesos (A y B) que se ejecutan a la vez.

El proceso "A" se conecta a internet y descarga un fichero de datos.

El proceso "B" abre el fichero descargado por "A" y lo procesa.

El proceso "B" no puede hacer el procesamiento del fichero hasta que "A" lo haya descargado.

Si se lanzan los dos, B tendrá que esperar a que A lo descargue. B es dependiente de A.

# Sincronización de procesos – Semáforos

Para sincronizar procesos se puede usar el concepto de "semáforo". Contador que controla cuantos procesos pueden acceder a un recurso compartido.

Analogía:

- Un baño público, en el que hay 3 cabinas, y hay un contador de cabinas libres (el semáforo).
- Cada vez que alguien entra disminuye el semáforo en 1 (resta 1), y cada cuando sale aumenta el semáforo en 1 (suma 1).
- Si el contador vale 0, nadie entra (significa que están todas ocupadas). Hay que esperar. Cómo gestionar la cola es otro asunto distinto...

# Sincronización de procesos – Semáforos

Java no tiene palabras reservadas para sincronizar procesos (para hilos sí). Se usan recursos externos a los procesos para sincronizarlos:

- Ficheros como semáforos – Usando la clase File.
- Bloqueo de ficheros (FileLock de Java NIO).

Además, podrían usarse otros mecanismos como:

- *Pipes (comunicación por E/S estándar entre procesos).*
- *Sockets (los veremos más adelante).*
- Otra fuente de datos externa (BD).

# Semáforos con File

Funcionamiento:

- Un proceso crea un fichero para indicar que un recurso está ocupado.
- Otro proceso espera hasta que el fichero desaparezca o cambie.
- Es un mecanismo muy rudimentario, que si no se implementa correctamente puede dar lugar a un funcionamiento incorrecto.

# Semáforos con File – Incorrecto

Ejemplo de uso de File que no funciona del todo correctamente:

```
File lock = new File("lock.tmp");

while (lock.exists()) { // Verificar si el semáforo existe
    Thread.sleep(100);   // Mientras exista, esperar hasta que esté disponible.
}

lock.createNewFile();      // "Adquirir" el uso del recurso.
// Usar el recurso: escribir en otro fichero, usar un servicio, etc.
lock.delete();       // Liberar recurso
```

Este uso de File no es correcto. Dos procesos pueden llegar a lock.exists() a la vez, y si los dos obtienen acceso, se produce acceso concurrente.

Ver ejemplo en repositorio (Ejemplo2).

# Semáforos con File – Mejorado

Se puede mejorar usando directamente createNewFile, y evaluando la salida de esta llamada, que devuelve false si ya existe el fichero:

```
File lock = new File("lock.tmp");
while (!lock.createNewFile()) {   // Intentar tomar control del recurso
    Thread.sleep(100);        // Esperar un tiempo antes de volver a intentarlo
}
try {
    // Usar el recurso: escribir en otro fichero, usar un servicio, etc.
} finally {
    lock.delete();            // Liberar el recurso
}
```

Ver ejemplo en repositorio (Ejemplo3).

# Semáforos con FileChannel

El uso de File, en las dos versiones, requiere espera activa, hay que dormir el proceso entre intentos de acceso. La clase FileChannel, de Java NIO, tiene un método "lock" que permite realizar un bloqueo real del proceso.

```
File lockFile = new File("lock.tmp");
lockFile.createNewFile;  // Crear el fichero sobre el que se hará el lock
try (FileOutputStream fos = new FileOutputStream(lockFile);
      FileChannel channel = fos.getChannel(); FileLock lock = channel.lock()) {
     // Hacer lo que se desee hacer.
} finally {
     if (lockFile.exists()) lockFile.delete();
}
```

Ver ejemplo en repositorio (Ejemplo4).

# Semáforos con FileChannel

El proceso, al llegar al try(), se bloquea si ya había otro proceso usándolo.

Cuando sale del try, se "libera" el semáforo automáticamente (el objeto FileLock es autoclosable), para que otro proceso pueda acceder.

Esta es la forma recomendada para hacer el bloqueo entre procesos.

Ojo, lo ideal es usar un objeto / fichero específico para el bloqueo.

Si se intenta usar el mismo objeto para bloquear y para escribir o si se usan streams con buffer, no funcionará correctamente, porque las operaciones sobre PrintWriter y BufferedWritter no se sincronizan bien.

Ver ejemplo en repositorio (ejemplo5).